



Desenvolvimento de Software

Back-End / Front-End

Disciplina: Desenvolvimento de Software (Back-End/Front-End)

Professor: Allan da Silva



Aula 01

Introdução ao Desenvolvimento Web



O que é Desenvolvimento de Software?

- Processo de **criação, projeto, implementação e manutenção** de software;
- Envolve diversas etapas: análise, design, codificação, testes e implantação;
- O objetivo é resolver problemas ou atender necessidades por meio de sistemas computacionais;
- Pode resultar em aplicações desktop, mobile, web, embarcadas, entre outras.



O que é uma Aplicação Web?

- Software que funciona **no navegador** do usuário;
- Não necessita de instalação local;
- Acessível de qualquer dispositivo com internet;
- Exemplos: redes sociais, e-mails, sistemas de gestão, lojas virtuais.



Evolução da Web

Geração	Características
Web 1.0	Páginas estáticas, somente leitura
Web 2.0	Conteúdo dinâmico, interação do usuário, redes sociais
Web 3.0	Inteligência artificial, descentralização, semântica



Cliente e Servidor



- **Cliente:** navegador do usuário (Chrome, Firefox, Edge...);
- **Servidor:** máquina que processa as requisições e envia respostas;
- A comunicação acontece por meio do protocolo **HTTP/HTTPS**.



Aplicações Web Modernas

- **SPA** (Single Page Application): carrega uma única página e atualiza dinamicamente;
- **PWA** (Progressive Web App): experiência semelhante a aplicativo nativo;
- **SSR** (Server-Side Rendering): renderização no servidor para melhor performance;
- **API-First**: back-end expõe APIs, front-end consome de forma independente.





Aula 02

O que é Front-End?



Conceito de Front-End

- É a parte da aplicação que o **usuário vê e interage** diretamente;
- Responsável pela **interface visual** e pela **experiência do usuário (UX)**;
- Roda no **navegador** do cliente;
- É a "fachada" da aplicação web.



Responsabilidades do Front-End

- Estruturar o conteúdo da página;
- Estilizar a aparência visual;
- Implementar interações e comportamentos;
- Consumir dados do back-end;
- Garantir responsividade e acessibilidade.



HTML — Estrutura

- **HyperText Markup Language**
- Define a **estrutura** e o **conteúdo** da página;
- Elementos: títulos, parágrafos, links, imagens, formulários, tabelas;
- Utiliza tags semânticas: `<header>` , `<nav>` , `<main>` , `<footer>` .

```
<h1>Olá, mundo!</h1>  
<p>Este é um parágrafo.</p>
```



CSS — Estilo

- **Cascading Style Sheets**
- Define a **aparência visual** da página;
- Controla cores, fontes, espaçamentos, layouts;
- Pode ser aplicado inline, interno ou externo.

```
h1 {  
  color: #C22820;  
  font-size: 2rem;  
}
```



JavaScript — Comportamento

- Linguagem de **programação** do navegador;
- Adiciona **interatividade** e **dinamismo**;
- Manipula o DOM (Document Object Model);
- Pode fazer requisições assíncronas ao servidor.

```
document.querySelector("h1").addEventListener("click", () => {  
    alert("Clicou no título!");  
});
```

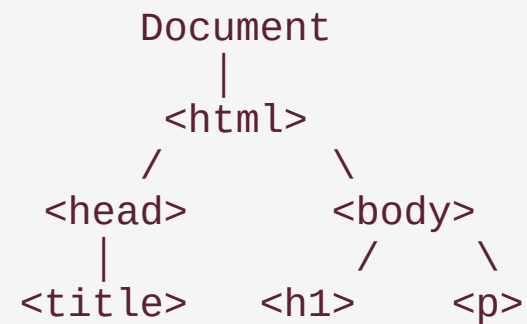


Interface e Experiência do Usuário

- **UI (User Interface):** design visual, botões, cores, tipografia;
- **UX (User Experience):** facilidade de uso, navegação intuitiva, acessibilidade;
- Um bom front-end combina **UI agradável** com **UX eficiente**.



DOM — Document Object Model



- Representação **estruturada** do documento HTML na memória;
- Permite que o JavaScript **leia, altere, adicione ou remova** elementos;
- É a ponte entre HTML e JavaScript.





Aula 03

O que é Back-End?



Conceito de Back-End

- É a parte da aplicação que roda **no servidor**;
- Responsável pelo **processamento de dados**, **regras de negócio** e **comunicação com o banco de dados**;
- O usuário **não vê** diretamente o back-end;
- É o "cérebro" da aplicação.



Servidor

- Máquina (física ou virtual) que **recebe requisições** e **envia respostas**;
- Pode ser local (desenvolvimento) ou na nuvem (produção);
- Exemplos de servidores: Apache, Nginx, Node.js, Tomcat;
- Funciona 24/7, aguardando requisições dos clientes.



Regras de Negócio

- Definem **como a aplicação funciona**;
- Exemplos:
 - "O desconto só é aplicado para compras acima de R\$ 100";
 - "Somente administradores podem excluir usuários";
 - "O estoque não pode ficar negativo";
- São implementadas no back-end para garantir **segurança e consistência**.



Banco de Dados

- Armazena os **dados da aplicação** de forma persistente;
- **Relacionais:** MySQL, PostgreSQL, SQL Server, Oracle;
- **Não relacionais:** MongoDB, Redis, Firebase;
- O back-end faz operações de **CRUD**:
 - **C**reate — Criar
 - **R**ead — Ler
 - **U**ppdate — Atualizar
 - **D**eleite — Excluir



Linguagens de Programação Back-End

Linguagem	Exemplos de Frameworks
JavaScript	Express, NestJS, Fastify
Python	Django, Flask, FastAPI
Java	Spring Boot, Quarkus
PHP	Laravel, Symfony
C#	ASP.NET
Ruby	Rails, Sinatra
Go	Gin, Echo



Responsabilidades do Servidor

- Receber e processar requisições HTTP;
- Executar regras de negócio;
- Acessar e manipular o banco de dados;
- Autenticar e autorizar usuários;
- Retornar respostas (JSON, HTML, etc.);
- Garantir segurança, performance e escalabilidade.





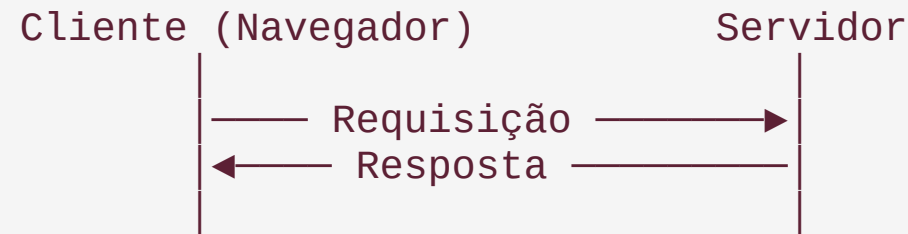
Aula 04

Como Front-End e Back-End se comunicam?



HTTP — HyperText Transfer Protocol

- Protocolo de comunicação **cliente ↔ servidor**;
- Baseado em **requisição e resposta**;
- Stateless: cada requisição é independente;
- HTTPS adiciona **criptografia** (segurança).





Request — Requisição

- Enviada pelo **cliente** ao servidor;
- Contém:
 - **Método** (GET, POST, PUT, DELETE...)
 - **URL** (endereço do recurso)
 - **Headers** (metadados: tipo de conteúdo, autenticação...)
 - **Body** (dados enviados, quando necessário)

```
GET /api/usuarios HTTP/1.1  
Host: exemplo.com  
Authorization: Bearer token123
```



Response — Resposta

- Enviada pelo **servidor** ao cliente;
- Contém:
 - **Status Code** (200, 404, 500...)
 - **Headers** (tipo de conteúdo, cache...)
 - **Body** (dados retornados)

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "nome": "Allan", "idade": 25 }
```



Métodos HTTP

Método	Finalidade	Exemplo
GET	Buscar dados	Listar usuários
POST	Criar recurso	Cadastrar usuário
PUT	Atualizar (completo)	Editar perfil
PATCH	Atualizar (parcial)	Alterar senha
DELETE	Remover recurso	Excluir conta



Status HTTP

Código	Significado	Exemplo
200	OK	Requisição bem-sucedida
201	Created	Recurso criado com sucesso
400	Bad Request	Dados inválidos
401	Unauthorized	Não autenticado
403	Forbidden	Sem permissão
404	Not Found	Recurso não encontrado
500	Internal Server Error	Erro no servidor



JSON — JavaScript Object Notation

- Formato **leve** de troca de dados;
- Baseado em pares **chave: valor**;
- Suportado por praticamente todas as linguagens.

```
{  
  "id": 1,  
  "nome": "Allan",  
  "email": "allan@email.com",  
  "ativo": true  
}
```



API — Application Programming Interface

- Conjunto de **endpoints** que o back-end expõe para o front-end consumir;
- **REST API**: utiliza métodos HTTP e URLs para manipular recursos;
- Cada endpoint representa uma **operação**:

GET	/api/usuarios	→ Listar todos
GET	/api/usuarios/1	→ Buscar por ID
POST	/api/usuarios	→ Criar
PUT	/api/usuarios/1	→ Atualizar
DELETE	/api/usuarios/1	→ Excluir





Aula 05

AJAX e Comunicação Assíncrona

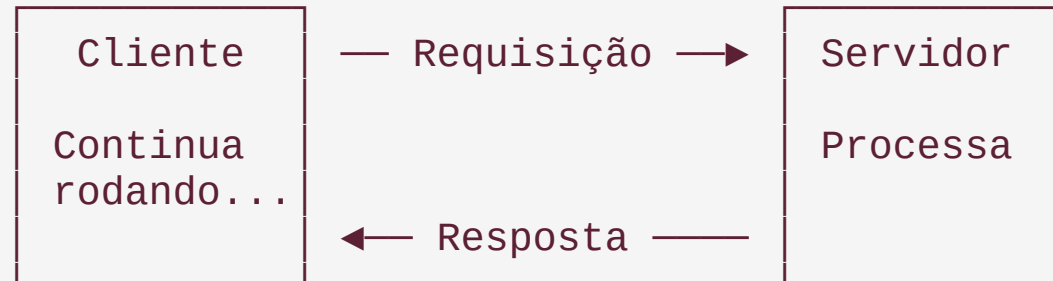


O que é AJAX?

- **Asynchronous JavaScript And XML;**
- Técnica para enviar e receber dados do servidor **sem recarregar a página;**
- Permite atualizar partes da página de forma dinâmica;
- Hoje, o formato mais utilizado é **JSON** (não mais XML).



Comunicação Assíncrona



- O navegador **não trava** enquanto espera a resposta;
- O JavaScript continua executando;
- Quando a resposta chega, uma **callback** ou **promise** é executada.



Requisições sem Recarregar a Página

- **Antes de AJAX:** cada ação recarregava a página inteira;
- **Com AJAX:** apenas os dados necessários são trocados;
- Resultado: aplicações mais **rápidas** e **fluidas**;
- Exemplos: autocomplete, likes, envio de formulários, infinite scroll.



fetch() — A API Moderna

```
fetch("https://api.exemplo.com/usuarios")
  .then(response => response.json())
  .then(dados => {
    console.log(dados);
  })
  .catch(erro => {
    console.error("Erro:", erro);
  });
```

- Substitui o antigo `XMLHttpRequest` ;
- Baseado em **Promises**;
- Suporta todos os métodos HTTP.



fetch() com async/await

```
async function buscarUsuarios() {  
  try {  
    const response = await fetch("https://api.exemplo.com/usuarios");  
    const dados = await response.json();  
    console.log(dados);  
  } catch (erro) {  
    console.error("Erro:", erro);  
  }  
}
```

- Sintaxe mais **limpa e legível**;
- Tratamento de erros com `try/catch`.



Tratamento de Respostas

```
async function criarUsuario(usuario) {  
  const response = await fetch("/api/usuarios", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify(usuario)  
  });  
  
  if (!response.ok) {  
    throw new Error(`Erro ${response.status}`);  
  }  
  
  return await response.json();  
}
```

- Verificar sempre `response.ok` ou `response.status` ;
- Tratar erros de rede e de servidor.



Exemplos Práticos de AJAX

- **Autocomplete de busca:** enquanto o usuário digita, sugestões aparecem;
- **Feed de notícias:** novos posts carregam automaticamente;
- **Carrinho de compras:** itens são adicionados sem recarregar;
- **Chat em tempo real:** mensagens aparecem instantaneamente;
- **Validação de formulário:** verifica disponibilidade de username em tempo real.





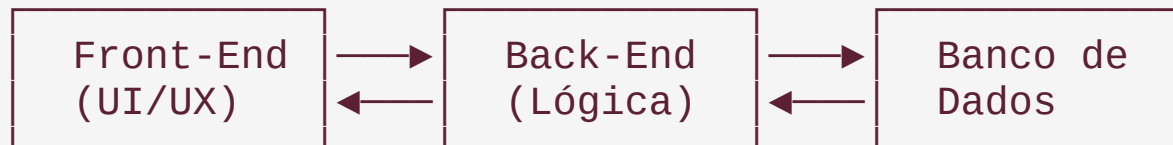
Aula 06

Boas Práticas em Front-End e Back-End



Separação de Responsabilidades

- **Front-End:** apresentação, interação, consumo de dados;
- **Back-End:** lógica de negócio, dados, segurança;
- Cada camada tem sua **responsabilidade clara**;
- Princípio **SoC** (Separation of Concerns).





Organização do Código

- Estrutura de pastas **lógica e consistente**;
- Nomes de arquivos e variáveis **descritivos**;
- Separação por **módulos** ou **camadas**:

```
src/  
├── controllers/  
├── services/  
├── repositories/  
├── models/  
├── routes/  
├── middlewares/  
└── utils/
```



Reutilização de Código

- Evite **duplicação** (DRY — Don't Repeat Yourself);
- Crie **funções** e **componentes** reutilizáveis;
- Utilize **bibliotecas** e **ferramentas** já existentes;
- Crie **helpers** e **utilitários** para tarefas comuns.



Segurança

- **Nunca confie** nos dados do front-end — valide tudo no back-end;
- Use **HTTPS** em produção;
- Proteja-se contra:
 - **SQL Injection** — use consultas parametrizadas;
 - **XSS** — sanitize inputs e outputs;
 - **CSRF** — use tokens de proteção;
- Armazene senhas com **hash** (bcrypt, argon2).



Validação

- Valide **no front-end** para melhor UX (feedback imediato);
- Valide **no back-end** para segurança (nunca confie no cliente);
- Exemplos de validação:
 - Campos obrigatórios;
 - Formato de e-mail;
 - Tamanho mínimo de senha;
 - Valores numéricos dentro de faixas.



Tratamento de Erros

```
try {  
    const resultado = await buscarDados();  
    exibirResultado(resultado);  
} catch (erro) {  
    exibirMensagem("Ocorreu um erro. Tente novamente.");  
    logErro(erro);  
}
```

- Sempre **trate erros** de forma elegante;
- Mensagens amigáveis para o **usuário**;
- Logs detalhados para o **desenvolvedor**;
- Nunca exponha informações sensíveis ao cliente.



Código Limpo e Manutenibilidade

- **Nomes claros** para variáveis e funções;
- **Funções pequenas** com responsabilidade única;
- **Comentários** apenas quando necessário;
- **Indentação** e formatação consistentes;
- **Code review** como prática de equipe;
- **Testes automatizados** para garantir qualidade.



Resumo da Unidade 1

Back-End / Front-End



Conceitos-Chave Aprendidos

- **Front-End:** HTML + CSS + JavaScript → interface do usuário;
- **Back-End:** servidor + regras de negócio + banco de dados;
- **HTTP:** protocolo de comunicação cliente ↔ servidor;
- **API:** conjunto de endpoints que o back-end expõe;
- **JSON:** formato de troca de dados;
- **AJAX:** comunicação assíncrona sem recarregar a página;
- **Boas práticas:** organização, segurança, validação e tratamento de erros.



Referências

Unidade 1 — Back-End / Front-End



Referências — Documentação e Sites

- **MDN Web Docs (Mozilla).** Referência completa para HTML, CSS e JavaScript. Disponível em: <https://developer.mozilla.org/pt-BR/>
- **W3Schools.** Tutoriais e referências de tecnologias web. Disponível em: <https://www.w3schools.com/>
- **W3C.** World Wide Web Consortium — padrões web. Disponível em: <https://www.w3.org/>
- **JSON.org.** Especificação oficial do formato JSON. Disponível em: <https://www.json.org/json-pt.html>



Referências — Livros

- CERQUEIRA, Maurício Samy. **JavaScript: Programando a Web com ES6+**. Rio de Janeiro: Casa do Código, 2017.
- FLANAGAN, David. **JavaScript: O Guia Definitivo**. 7ª ed. São Paulo: Novatec, 2020.
- DUCKETT, Jon. **HTML & CSS: Projete e Construa Websites**. Rio de Janeiro: Alta Books, 2014.
- DUCKETT, Jon. **JavaScript & jQuery: Projete e Construa Websites**. Rio de Janeiro: Alta Books, 2015.
- ROBBINS, Jennifer Nystrom. **Aprendendo Web Design**. 5ª ed. Rio de Janeiro: Alta Books, 2019.



Referências — Artigos e Artigos Científicos

- FIELDING, Roy T. **Architectural Styles and the Design of Network-based Software Architectures.** Doctoral Dissertation, University of California, Irvine, 2000. Disponível em: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- BERNERS-LEE, Tim. **Information Management: A Proposal.** CERN, 1989. Disponível em: <https://www.w3.org/History/1989/proposal.html>
- GARRETT, Jesse James. **Ajax: A New Approach to Web Applications.** Adaptive Path, 2005. Disponível em: <https://adaptivepath.org/ajax-a-new-approach-to-web-applications/>

